Git Basic Commands Troubleshooting SOP

Cognizant

Table of Contents

1. Git Basic Commands Troubleshooting 3
1.1 Description 3
1.2 Architecture Diagram 3
1.3 Lab Steps 4

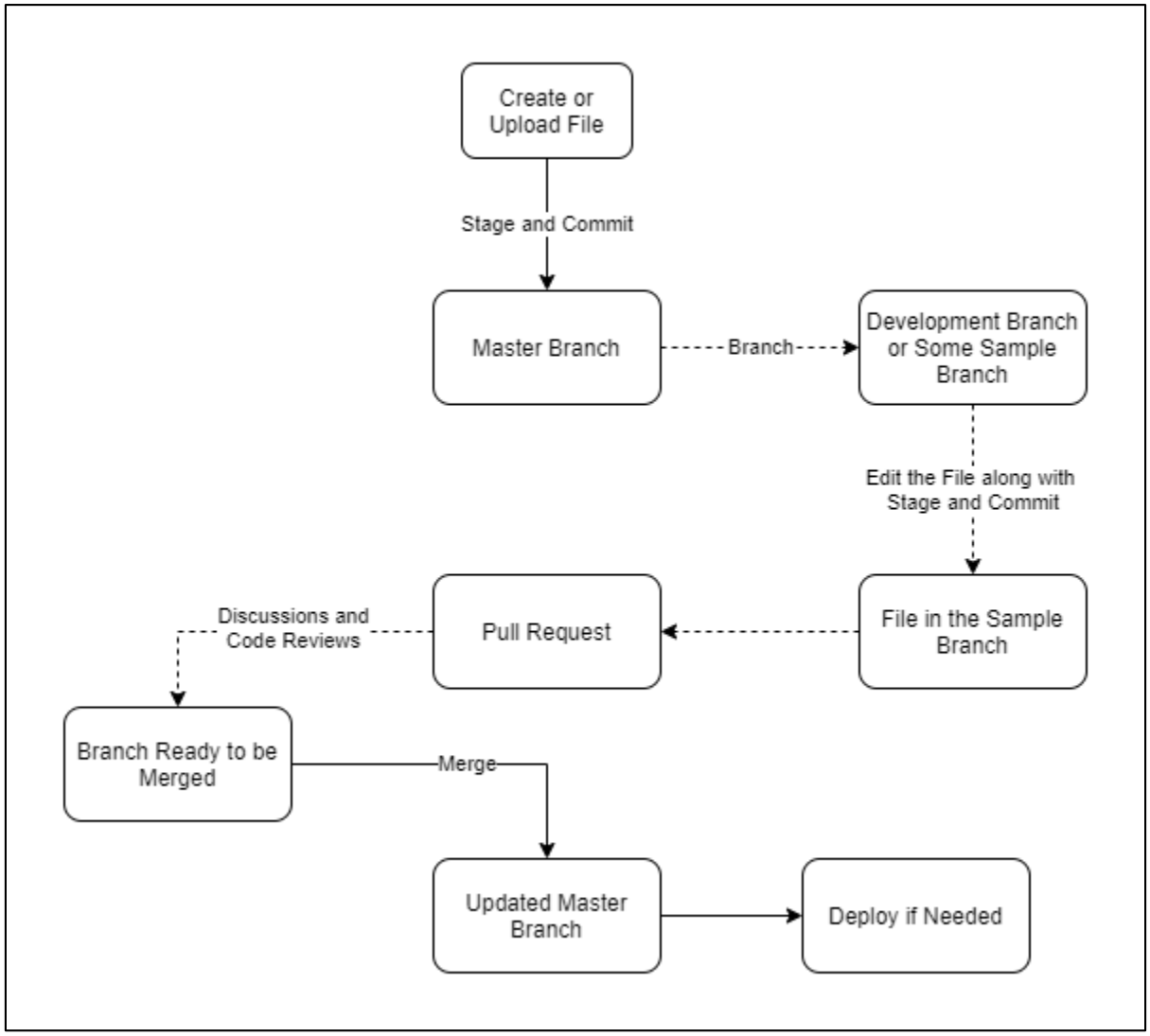
1. Git Basic Commands Troubleshooting

1.1 Description

Git is a Distributed Version Control System that helps keep track of code changes and maintains repositories. Once the flow of Git is understood, it is very easy to work and collaborate. In this usecase, the basic commands and potential troubleshooting options are discussed.

1.2 Architecture Diagram

The diagram below displays a visual representation of the underlying application architecture:



1.3 Lab Steps

Follow the steps outlined below to achieve the objective of this lab exercise:

1. Create an Ubuntu EC2 Instance. SSH into the instance using **Sessions Manager** option.
   Note: Make sure that the role **CCL-EC2-Role** is attached to the instance.
2. Change user to super user or Root User using the following command:

   ```
   sudo su
   ```

3. Change to root folder.
4. Update the Instance with the following command:

   ```
   apt update && apt upgrade -y
   ```

5. Install Git using the following command:

   ```
   apt-get install git
   ```

6. Create a folder with some name and change directory to it.
7. The following is the first command and this command initializes the folder to track the changes, so that at a later point in time, it can be pushed to either GitHub or CodeCommit etc:

   ```
   git init
   ```

8. Create a file in the folder. Any text file will do.
9. Once the file is created, enter the following command which will display information as to which files are tracked, which files are not tracked etc.

   ```
   git status
   ```

10. The next step in the flow is to stage the files. This will stage the created files or updated files for pushing. To perform this, enter the following command for all the files in the folder:

    ```
    git add .
    ```

    Or the following command for a single file:

    ```
    git add <filename>
    ```

11. Now enter the following command to see the status:

    ```
    git status
    ```

12. The next step is to commit the staged file with a commit message. This will let the users of the repository to see what the exact change was. Enter the following command:

    ```
    git commit -m "<Some Message>
    ```

13. You should find a prompt stating username and email are configured automatically. It is a best practice to include your original name and email ID so that the developers or those who use that repo knows who did what.
    To perform this enter the following commands:

    ```
    git config --global user.name <your-name>
    ```

    ```
    git config --global user.email <your-email>
    ```

14. To check these, enter the following command:

    ```
    git config --global --list
    ```

15. Execute the following command which is the Push command which will push the code to a remote repository like GitHub or CodeCommit etc:

    ```
    git push
    ```

    Note: This will throw an error message stating **No configured push destination**.
16. To handle this issue, if it is in GitHub, you need to create a repository and copy the clone link and use the following command. However this is out of scope of this topic:

    ```
    git remote add <some-name> <url>
    ```

    These are the basic commands that are used commonly while using Git.